

EGCN WebApp

Implementation Plan & Technical SRS
Marketing · Finance · CashFlow Platform

Stack: MongoDB · Express · React · Node.js (MERN)

Document Version: v1.0 — May 2026

Modules: 11 Modules · Full-Stack Implementation

Prepared For: EGCN Engineering Team

Table of Contents

00 Project Overview & Tech Stack

- Architecture Overview
- Tech Stack Decision
- Project Structure

01 Module 1 — Homepage (Public Landing Page)

- Components
- API Routes
- Implementation Plan

02 Module 2 — Authentication (Signup / Signin)

- Signup Flow
- Signin Flow
- JWT Strategy
- Schema

03 Module 3 — Dashboard

- Layout & Sidebar
- Stats Cards
- API Integration

04 Module 4 — Target : Daily Input

- Form Fields
- Business Logic
- Chart Generation

05 Module 5 — Target : Monthly Analysis

- Monthly Target Form
- Projection Engine
- Visuals

06 Module 6 — Insights & Reports

- Daily Report
- Monthly Report
- Annual Report
- BEP Report

07 Module 7 — Workspace (Communication UI)

- Chat
- Media Sharing
- Video Calls
- Calendar

08 Module 8 — Plans & Pricing

- Plan Gating

- Razorpay Integration
- Webhook

09 Module 9 — Admin Panel

- User Management
- Expert Assignment
- Plan Control

10 Module 10 — Notification & Suggestion Engine

- Static Message Pool
- Trigger Logic

11 Module 11 — Deployment & DevOps

- Environment Setup
- CI/CD
- Hosting Strategy

12 Master Implementation Timeline

- Sprint Plan
- Effort Estimate

Project Overview & Tech Stack

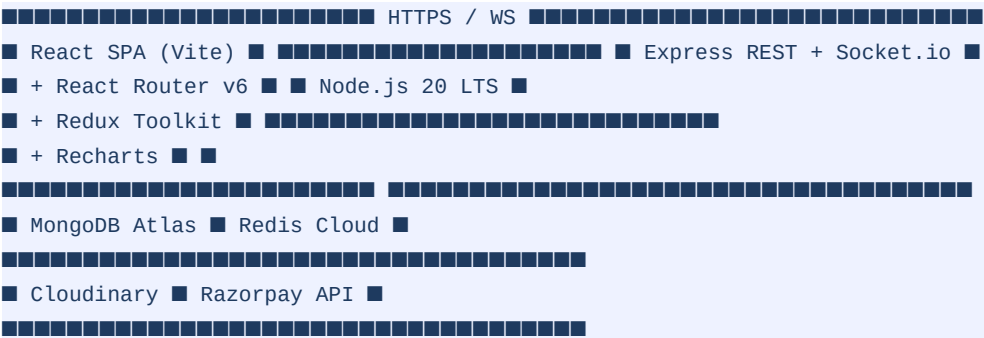
Full-Stack Architecture · MERN · Real-Time · Payments

What is EGCN WebApp?

EGCN WebApp is a B2B SaaS consulting platform that digitises the marketing, finance, and cashflow advisory services of EGCN Consulting. Businesses register on the platform, set sales and billing targets, track daily performance, receive AI-style suggestions, view rich analytics reports, and communicate directly with EGCN experts through an integrated Workspace. A tiered subscription model (Razorpay) unlocks premium features including expert chat, video calls, media sharing, and site visits.

Architecture Overview

Frontend (React SPA) communicates with the **Backend (Express REST API)** over HTTPS. The backend connects to **MongoDB Atlas** for persistence, **Redis** for session caching & rate-limiting, and **Socket.io** for real-time chat. **Razorpay** handles subscription payments. Media files go to **Cloudinary / AWS S3**. Video calls use **WebRTC + PeerJS** or a hosted service (Daily.co).



Tech Stack Decision Matrix

React 18 + Vite	Fast SPA, code splitting, HMR. React Router v6 for routing.
Redux Toolkit	Global state: auth, user data, plan tier, sidebar state.
TailwindCSS	Utility-first styling. No custom CSS overhead. Dark-mode ready.
Recharts	D3-based charting library. Line, Bar, Area, Pie charts for Insights.

Node.js 20 LTS	Non-blocking I/O. Perfect for real-time + REST combined.
Express.js 4	Minimal, unopinionated REST framework. Middleware-based.
MongoDB Atlas	Document DB. Flexible schema for business data, targets, reports.
Mongoose 8	Schema validation, virtuals, populate, indexes.
Socket.io 4	Bi-directional real-time events for Chat & Workspace.
Redis (Upstash)	JWT blacklist, rate limiting, session store, suggestion cache.
JWT + bcrypt	Stateless auth. Access token (15m) + Refresh token (7d).
Razorpay	Indian payment gateway. Subscription plans + webhook verification.
Cloudinary	Image/video upload & CDN for media sharing in Workspace.
WebRTC / PeerJS	Peer-to-peer video call between expert and client.
Nodemailer + SendGrid	Transactional emails: OTP, welcome, invoice, subscription.
Multer	File upload middleware. 25MB limit. MIME type validation.
Zod	Runtime schema validation for all API request bodies.
Winston + Morgan	Structured logging. Morgan for HTTP, Winston for app-level.
Docker + Nginx	Containerised deployment. Nginx as reverse proxy + SSL terminator.
GitHub Actions	CI/CD pipeline: lint → test → build → deploy.

Full Project Directory Structure

```

egcn-webapp/
├── client/ # React Frontend (Vite)
│   ├── public/
│   └── src/
│       ├── assets/ # Images, fonts, icons
│       ├── components/ # Reusable UI components
│       │   ├── common/ # Button, Modal, Badge, Loader
│       │   ├── charts/ # LineChart, BarChart, DonutChart
│       │   ├── layout/ # Sidebar, Header, Footer
│       │   └── workspace/ # Chat, VideoCall, Calendar
│       ├── pages/ # Route-level page components
│       │   ├── Home/ # Landing page sections
│       │   ├── Auth/ # Signup, Signin
│       │   ├── Dashboard/ # Main dashboard
│       │   ├── Target/ # Daily & Monthly target
│       │   ├── Insights/ # Reports (Daily/Monthly/Annual)
│       │   └── Workspace/ # Chat + Video + Calendar
│       ├── Plans/ # Pricing + Payment
│       └── store/ # Redux Toolkit slices

```

```

■ ■ ■ ■ ■ authSlice.js
■ ■ ■ ■ ■ targetSlice.js
■ ■ ■ ■ ■ insightsSlice.js
■ ■ ■ ■ ■ workspaceSlice.js
■ ■ ■ ■ ■ hooks/ # Custom React hooks
■ ■ ■ ■ ■ services/ # Axios instances + API calls
■ ■ ■ ■ ■ utils/ # Formatters, validators, helpers
■ ■ ■ ■ ■ constants/ # Static suggestions (50 msgs), plans
■ ■ ■ ■ ■ router/ # React Router config + ProtectedRoute
■ ■ ■ ■ ■ App.jsx
■ ■ ■ ■ ■ main.jsx
■ ■ ■ ■ ■ .env.local
■ ■ ■ ■ ■ vite.config.js
■ ■ ■ ■ ■ tailwind.config.js
■
■ ■ ■ ■ ■ server/ # Express Backend
■ ■ ■ ■ ■ src/
■ ■ ■ ■ ■ config/ # DB, Redis, Cloudinary, Razorpay config
■ ■ ■ ■ ■ controllers/ # Route handler logic
■ ■ ■ ■ ■ auth.controller.js
■ ■ ■ ■ ■ target.controller.js
■ ■ ■ ■ ■ insights.controller.js
■ ■ ■ ■ ■ workspace.controller.js
■ ■ ■ ■ ■ payment.controller.js
■ ■ ■ ■ ■ admin.controller.js
■ ■ ■ ■ ■ models/ # Mongoose schemas
■ ■ ■ ■ ■ User.model.js
■ ■ ■ ■ ■ DailyEntry.model.js
■ ■ ■ ■ ■ MonthlyTarget.model.js
■ ■ ■ ■ ■ Message.model.js
■ ■ ■ ■ ■ Meeting.model.js
■ ■ ■ ■ ■ Subscription.model.js
■ ■ ■ ■ ■ routes/ # Express routers
■ ■ ■ ■ ■ middleware/ # Auth guard, role guard, rate-limit
■ ■ ■ ■ ■ services/ # Business logic layer
■ ■ ■ ■ ■ suggestion.service.js
■ ■ ■ ■ ■ report.service.js
■ ■ ■ ■ ■ bep.service.js
■ ■ ■ ■ ■ sockets/ # Socket.io event handlers
■ ■ ■ ■ ■ utils/ # Logger, email, validators
■ ■ ■ ■ ■ app.js # Express app + socket init
■ ■ ■ ■ ■ server.js # Entry point
■ ■ ■ ■ ■ .env
■
■ ■ ■ ■ ■ docker-compose.yml
■ ■ ■ ■ ■ nginx.conf
■ ■ ■ ■ ■ README.md

```

Homepage — Public Landing Page

Marketing Surface · SEO · Testimonials · CTA

Section Breakdown

Field	Type	Required	Notes
Site Header	React Component	Yes	Logo (left) + Login button (right). Sticky, transparent → solid on scroll.
Hero Section	React Component	Yes	Bold tagline, secondary tagline (grey), two red CTA buttons: 'Get Started' & 'Pricing'.
Sub Hero Tiles	React Component	Yes	3 service cards: Marketing, Finance, CashFlow. Each has readmore (bottom-right click).
Myth Breaker	React Component	Yes	Tab-format, 3–4 myth/fact points. Expand via 'Read More'. Animated tab switching.
Testimonials	React Component	Yes	Carousel (min 3 items), slow auto-scroll. Each: client name, logo, review text.
Founder Philosophy	React Component	Yes	Full-bleed bg image (low opacity). Quote in chest area. Founder name: Param Joshi.
Footer	React Component	Yes	Left: logo + tagline + socials. Right: About, Contact, Subscribe. Bottom: legal links.

Component File Structure

```
src/pages/Home/
■■■ index.jsx # Page entry, composes all sections
■■■ SiteHeader.jsx # Sticky nav bar
■■■ HeroSection.jsx # Hero + CTA buttons
■■■ SubHeroTiles.jsx # 3 service tiles
■■■ MythBreaker.jsx # Tab accordion
■■■ TestimonialsCarousel.jsx # Auto-scroll carousel
■■■ FounderPhilosophy.jsx # Founder quote section
■■■ Footer.jsx # Footer layout

src/constants/
■■■ testimonials.js # Array of testimonial objects
■■■ mythBreaker.js # Array of myth/fact pairs
```

API Routes (Homepage needs)

Method	Route	Description	Auth
GET	/api/testimonials	Fetch all active testimonials	Public
GET	/api/services	Fetch service descriptions	Public
POST	/api/newsletter	Subscribe email to newsletter	Public

Implementation Plan

Phase	Task	Duration	Priority
Setup	Install Vite + React + TailwindCSS + React Router	0.5d	Critical
UI	Build SiteHeader with sticky scroll + Login button	0.5d	High
UI	Build HeroSection with animated tagline + CTA buttons	0.5d	High
UI	Build 3 SubHero service tiles with readmore modal	0.5d	High
UI	Build MythBreaker tab accordion with expand	0.5d	Medium
UI	Build Testimonials auto-carousel (Swiper.js or custom)	1d	High
UI	Build FounderPhilosophy with bg-image overlay	0.5d	Medium
UI	Build responsive Footer with all links	0.5d	Medium
Backend	GET /api/testimonials + POST /api/newsletter	0.5d	Low
Polish	Mobile responsiveness + SEO meta tags + OG images	1d	High

Authentication — Signup & Signin

JWT · Bcrypt · Captcha · Role-Based Access

User Schema (MongoDB)

Field	Type	Required	Notes
email	String	Yes	Unique, lowercase, indexed. Used as login identifier.
password	String	Yes	Bcrypt hashed (salt rounds: 12). Never returned in API response.
businessType	String	Yes	Dropdown value: Retail / Service / Manufacturing / Other.
businessName	String	Yes	Name of the registered business.
businessScale	String	Yes	Enum: '0-5L' '5-15L' '15L+' (annual turnover range).
address	String	Yes	Street address.
city	String	Yes	City of operation.
state	String	Yes	Indian state — dropdown or text.
phone	String	Yes	10-digit Indian mobile number. Validated via regex.
role	String	Yes	Enum: 'client' 'expert' 'admin'. Default: 'client'.
plan	String	No	Enum: 'free' 'business' 'pro' 'vip'. Default: 'free'.
planExpiry	Date	No	Subscription expiry date set by Razorpay webhook.
isVerified	Boolean	No	Email verification flag. Default: false.
refreshToken	String	No	Hashed refresh token stored server-side for rotation.
createdAt	Date	Auto	Mongoose timestamps.

API Routes

Method	Route	Description	Auth
POST	/api/auth/signup	Register new business user	Public
POST	/api/auth/signin	Login, returns access + refresh tokens	Public

POST	/api/auth/refresh	Rotate access token using refresh token	Public
POST	/api/auth/logout	Blacklist refresh token in Redis	Auth
POST	/api/auth/verify-email	Verify email via OTP	Public
POST	/api/auth/forgot-password	Send password reset OTP email	Public
POST	/api/auth/reset-password	Reset password using OTP	Public
GET	/api/auth/me	Get logged-in user profile	Auth

JWT Strategy

Access Token	Short-lived (15 minutes). Sent in Authorization: Bearer header. Stored in memory (Redux).
Refresh Token	Long-lived (7 days). HttpOnly cookie. Hashed copy stored in MongoDB User doc.
Rotation	On every /refresh call, old refresh token is invalidated and new pair is issued.
Blacklist	Logout blacklists the refresh token in Redis with TTL = token expiry time.
Guards	authenticate middleware verifies access token. authorise('admin') checks role.

Signup Flow (Step-by-Step)

1. Client submits form → Zod validates all fields server-side.
2. Check if email already exists → return 409 if duplicate.
3. Hash password with bcrypt (12 rounds).
4. Create User document in MongoDB.
5. Send verification OTP email via Nodemailer + SendGrid.
6. Return 201 with message: 'Verify your email to continue'.
7. User verifies OTP → isVerified = true → redirect to Dashboard.

Implementation Plan

Phase	Task	Duration	Priority
Backend	User.model.js with all fields + Mongoose indexes	0.5d	Critical
Backend	POST /auth/signup with Zod validation + bcrypt	0.5d	Critical
Backend	POST /auth/signin with JWT pair generation	0.5d	Critical

Backend	POST /auth/refresh + /auth/logout with Redis blacklist	0.5d	High
Backend	Email OTP via Nodemailer + SendGrid template	1d	High
Frontend	Signup multi-field form with client-side validation	1d	Critical
Frontend	Signin form + error handling + redirect logic	0.5d	Critical
Frontend	ProtectedRoute HOC + Redux auth slice + axios interceptor	1d	Critical
Frontend	OTP verification page	0.5d	High
Testing	Auth flow E2E: signup → verify → login → refresh → logout	1d	High

Dashboard — Post-Login Overview

Stats Cards · Sidebar · Site Header · Greeting

Layout Architecture

Layout	CSS Grid: sidebar (240px fixed) + main content area (flex-grow).
Site Header	Top bar inside main area. Shows user greeting + notification bell + avatar dropdown.
Greeting	Dynamic: 'Hi [BusinessName], Good Morning/Afternoon/Evening' based on system time.
Overall Row	3 blue stat cards: Total Bills (all time), Total Sales (all time), Active Plan.
Daily Row	3 red stat cards: Today's Bills, Today's Sales, Today's Target %.

Sidebar Navigation Tree

```
Sidebar (root)
  ■■■ Overview → /dashboard
  ■■■ Target
    ■ ■■■ Target Analysis → /dashboard/target/analysis
    ■ ■■■ Monthly Target → /dashboard/target/monthly
    ■ ■■■ Daily Input → /dashboard/target/daily
  ■■■ Insights
    ■ ■■■ Daily Report → /dashboard/insights/daily
    ■ ■■■ Monthly Report → /dashboard/insights/monthly
    ■ ■■■ Annual Report → /dashboard/insights/annual
  ■■■ Consult Agency → /dashboard/workspace
```

API Routes

Method	Route	Description	Auth
GET	/api/dashboard/summary	Overall + daily stats for stat cards	Auth
GET	/api/dashboard/user	User profile, plan, business info	Auth

Dashboard Summary Response Shape

```
{
  "overall": {
    "totalBills": 1240,
    "totalSales": 5200000,
    "activePlan": "pro"
  },
  "daily": {
    "todayBills": 18,
    "todaySales": 45000,
    "targetAchievedPercent": 72
  }
}
```

Implementation Plan

Phase	Task	Duration	Priority
Frontend	Sidebar component with nested nav + active state highlighting	1d	Critical
Frontend	SiteHeader with greeting logic + avatar dropdown + notifications	0.5d	High
Frontend	3x Overall stat cards (blue) + 3x Daily stat cards (red)	0.5d	High
Backend	GET /api/dashboard/summary aggregation query from DailyEntry	1d	Critical
Frontend	Wire stat cards to Redux store + loading skeleton	0.5d	High
UI	Mobile-responsive sidebar collapse (hamburger on mobile)	0.5d	Medium

Target — Daily Input & Analysis

Daily Entry · Graph Generation · Suggestion Engine

DailyEntry Schema (MongoDB)

Field	Type	Required	Notes
userId	ObjectId	Yes	Ref: User. Indexed for fast per-user queries.
date	Date	Yes	Date of entry. Compound unique index with userId.
bills	Number	Yes	Number of bills/transactions made today.
sales	Number	Yes	Total sales amount (INR) for today.
targetBills	Number	No	Derived from MonthlyTarget. Required daily bills.
targetSales	Number	No	Derived from MonthlyTarget. Required daily sales.
achievedPct	Number	No	Computed virtual: (sales / targetSales) * 100.
suggestion	String	No	One of 50 static messages selected at entry time.
createdAt	Date	Auto	Mongoose timestamps.

API Routes

Method	Route	Description	Auth
POST	/api/target/daily	Submit today's bills + sales	Auth
GET	/api/target/daily	Get today's entry + computed analysis	Auth
GET	/api/target/daily/history	Last 30 days entries for chart	Auth
GET	/api/target/daily/suggest	Return today's suggestion message	Auth

Business Logic — Daily Analysis

Required Daily Bills	$\text{monthlyTargetBills} \div \text{workingDaysInMonth}$
Required Daily Sales	$\text{monthlyTargetSales} \div \text{workingDaysInMonth}$

Achievement %	$(\text{todaySales} \div \text{requiredDailySales}) \times 100$
Suggestion Trigger	Randomly selected from 50-message pool, weighted by achievement %
Charts Generated	1) Today's Bills vs Target (Bar), 2) Today's Sales vs Target (Bar), 3) 30-day trend (Line)

Suggestion Engine (50 Static Messages)

The suggestion engine picks a message from a pool of 50 pre-written advisory messages stored in `src/constants/suggestions.js`. The selection is semi-random but weighted by performance tier:

- < 50% achieved → motivational/urgency messages
- 50–80% achieved → on-track encouragement messages
- > 80% achieved → celebration + push-harder messages

The 'Upgrade Plan' button below the suggestion is shown to free-tier users and links to the Plans & Pricing page.

Implementation Plan

Phase	Task	Duration	Priority
Backend	DailyEntry.model.js with compound unique index (userId + date)	0.5d	Critical
Backend	POST /api/target/daily with working-days calculation logic	1d	Critical
Backend	GET /api/target/daily/history — 30-day aggregation pipeline	1d	High
Frontend	Daily Input form (bills + sales) with date picker	0.5d	Critical
Frontend	Bar chart: Today Bills vs Target (Recharts BarChart)	0.5d	High
Frontend	Bar chart: Today Sales vs Target (Recharts BarChart)	0.5d	High
Frontend	Suggestion display card + Upgrade Plan CTA	0.5d	High
Frontend	30-day trend Line chart on Target Analysis page	1d	High
Backend	Suggestion service: 50 messages pool + tier-weighted selector	0.5d	Medium

Target — Monthly Analysis

Monthly Goal Setting · Projection · Visual Tracking

MonthlyTarget Schema (MongoDB)

Field	Type	Required	Notes
userId	ObjectId	Yes	Ref: User.
month	Number	Yes	1–12. Compound unique index with userId + year.
year	Number	Yes	Full year e.g. 2026.
targetBills	Number	Yes	User's monthly bills goal.
targetSales	Number	Yes	User's monthly sales goal (INR).
requiredDaily	Object	No	Computed: {bills: N, sales: N} from target ÷ working days.
achievedBills	Number	No	Sum of DailyEntry.bills for this month (aggregated).
achievedSales	Number	No	Sum of DailyEntry.sales for this month (aggregated).
projectedSales	Number	No	Linear projection: (achievedSales/daysElapsed) × totalDays.

API Routes

Method	Route	Description	Auth
POST	/api/target/monthly	Set or update monthly target	Auth
GET	/api/target/monthly/:month	Get monthly target + achieved + projected	Auth
GET	/api/target/monthly/all	Get all months for annual chart	Auth

Computed Visuals on Monthly Page

Chart 1 — Daily Comparison	Bar chart: Required daily bills/sales vs actual daily bills/sales for each day in month.
Chart 2 — Monthly Progress	Donut/ring chart: Achieved sales vs Remaining sales vs Target.

Chart 3 — Projection	Line chart: Actual trend (solid) + Projected trend (dashed) to end of month.
Summary Cards	3 cards: Total Achieved, Remaining, Projected End-of-Month.

Implementation Plan

Phase	Task	Duration	Priority
Backend	MonthlyTarget.model.js with compound unique index	0.5d	Critical
Backend	POST /api/target/monthly with validation + requiredDaily calc	0.5d	Critical
Backend	GET monthly with MongoDB \$lookup aggregation from DailyEntry	1d	Critical
Backend	Projection algorithm: linear regression on days elapsed	1d	High
Frontend	Monthly Target form (bills + sales inputs)	0.5d	High
Frontend	Required daily breakdown display	0.5d	High
Frontend	Donut chart — achieved vs remaining vs target	0.5d	High
Frontend	Projection Line chart (solid + dashed)	1d	High
Frontend	3 summary stat cards	0.5d	Medium

Insights & Reports

Daily · Monthly · Annual · BEP Analysis

Report Types

Daily Report	Today's bills + sales. Actual vs Expected bar charts. Suggestion. Upgrade CTA.
Monthly Report	Month-level bills + sales. Achieved vs target charts. Month-over-month trend.
Annual Report	Full-year view. Month-by-month performance. Best/worst month highlight.
BEP Report	Break-Even Point analysis: fixed costs, variable costs, contribution margin, BEP units.

BEP Report Logic

The Break-Even Point (BEP) analysis requires additional one-time inputs from the user:

- Fixed Costs (monthly rent, salaries, overheads)
- Variable Cost per Unit
- Selling Price per Unit

BEP Formula:

Contribution Margin (CM) = Selling Price - Variable Cost per Unit

BEP (units) = Fixed Costs ÷ CM

BEP (sales value) = BEP units × Selling Price

Margin of Safety = Actual Sales - BEP Sales

MoS % = (Margin of Safety ÷ Actual Sales) × 100

API Routes

Method	Route	Description	Auth
GET	/api/insights/daily	Today's report data	Auth
GET	/api/insights/monthly/:month	Monthly report aggregation	Auth

GET	/api/insights/annual/:year	Annual report by month	Auth
POST	/api/insights/bep	Submit BEP inputs, get analysis	Auth
GET	/api/insights/bep/history	Past BEP analyses	Auth

Implementation Plan

Phase	Task	Duration	Priority
Backend	report.service.js — daily/monthly/annual aggregation pipelines	2d	Critical
Backend	bep.service.js — BEP calculation engine	1d	High
Frontend	Daily Report page with 2 charts + suggestion card	1d	Critical
Frontend	Monthly Report page with bar + line charts	1d	High
Frontend	Annual Report page — 12-month bar chart + summary cards	1d	High
Frontend	BEP Report page — input form + results cards + charts	1.5d	High
Frontend	PDF export button (jsPDF + html2canvas) for reports	1d	Medium

Workspace — Communication UI

Chat · Media Sharing · Video Calls · Calendar

Feature Breakdown

Chat Window	Real-time text messaging between client and assigned EGCN expert. Socket.io rooms.
Media Sharing	Upload images, PDFs, documents. Cloudinary CDN. 25MB limit. Preview in-chat.
Video Calls	WebRTC peer-to-peer video. PeerJS for signaling. Screen-share option.
Calendar	Google-Calendar-style UI. Schedule meetings. CRUD on events. Expert + client shared view.
Reports Auto-sync	When expert shares a PDF/report in chat, it auto-appears in client's Insights section.

Message Schema (MongoDB)

Field	Type	Required	Notes
roomId	String	Yes	Unique room per client-expert pair. Format: {clientId}_{expertId}.
senderId	ObjectId	Yes	Ref: User. Who sent the message.
receiverId	ObjectId	Yes	Ref: User. Who receives the message.
type	String	Yes	Enum: 'text' 'image' 'document' 'video'.
content	String	No	Text content (for type = 'text').
fileUrl	String	No	Cloudinary URL (for media types).
fileName	String	No	Original filename for download.
isReport	Boolean	No	If true, auto-sync to client's Insights. Default: false.
seenAt	Date	No	When receiver opened the message. For 'seen' ticks.

createdAt	Date	Auto	Mongoose timestamps.
-----------	------	------	----------------------

Meeting Schema (MongoDB)

Field	Type	Required	Notes
roomId	String	Yes	Shared room between client and expert.
title	String	Yes	Meeting title/subject.
scheduledAt	Date	Yes	Meeting date + time.
duration	Number	Yes	Duration in minutes (default: 60).
type	String	Yes	Enum: 'video' 'site_visit'. Site visit for Pro/VIP plan.
status	String	Yes	Enum: 'scheduled' 'completed' 'cancelled'.
meetingLink	String	No	PeerJS room ID or Daily.co link.

API Routes

Method	Route	Description	Auth
GET	/api/workspace/messages/:roomId	Get last 50 messages in room	Auth
POST	/api/workspace/messages	Send text message	Auth
POST	/api/workspace/upload	Upload media to Cloudinary	Auth
GET	/api/workspace/meetings	Get all meetings for user	Auth
POST	/api/workspace/meetings	Schedule a new meeting	Auth
PUT	/api/workspace/meetings/:id	Update meeting status	Auth
GET	/api/workspace/calendar	Get events for calendar view	Auth

Socket.io Events

join_room	Client emits on connect with roomId. Server joins socket to room.
send_message	Client emits with message payload. Server broadcasts to room.
receive_message	Server emits to room. All clients in room receive new message.
typing	Client emits when typing. Server relays to room (debounced).
seen	Client emits when message opened. Server updates seenAt in DB.
call_init	Client initiates video call. Server relays PeerJS peerId to partner.
call_accepted	Partner accepts. WebRTC handshake begins via PeerJS.

call_ended	Either party ends call. Server logs duration.
-------------------	---

Implementation Plan

Phase	Task	Duration	Priority
Backend	Message + Meeting models + Socket.io room management	1d	Critical
Backend	REST routes for messages + Cloudinary upload middleware	1d	Critical
Backend	Socket.io events: join, send, typing, seen	1d	Critical
Frontend	Chat UI: message list + input box + file upload button	2d	Critical
Frontend	Media preview in-chat (image thumbnail + doc icon)	1d	High
Backend	PeerJS signaling server or Daily.co integration for video	1d	High
Frontend	Video call UI: local + remote video tiles + controls bar	2d	High
Frontend	Calendar UI (react-big-calendar or FullCalendar)	1.5d	High
Backend	Auto-sync: if isReport=true → push to client Insights	0.5d	Medium
Testing	Multi-user chat + call E2E test	1d	High

Plans & Pricing — Razorpay Integration

Subscription Tiers · Webhook · Feature Gating

Pricing Plans

Free	Default plan. Access to Dashboard, Target, Insights (read-only). No Workspace.
Business Enhancement ■799/mo	Workspace unlocked: Chat + Media Sharing + Calendar. Prescription UI.
Pro Plan ■1499/mo	Everything in Business + Expert Site Visit scheduling.
VIP Plan ■2999/mo	Everything in Pro + Resource Utilisation consultation. Dedicated expert.

Subscription Schema (MongoDB)

Field	Type	Required	Notes
userId	ObjectId	Yes	Ref: User.
plan	String	Yes	Enum: 'free' 'business' 'pro' 'vip'.
razorpayOrderId	String	No	Razorpay order ID returned from create-order API.
razorpayPayId	String	No	Razorpay payment ID. Verified server-side.
razorpaySubId	String	No	Razorpay subscription ID for recurring billing.
amount	Number	Yes	Amount in paise (799*100 = 79900).
status	String	Yes	Enum: 'created' 'paid' 'failed' 'cancelled'.
startDate	Date	No	Subscription start date.
endDate	Date	No	Subscription end date (startDate + 30 days).
invoice	String	No	Cloudinary URL of generated invoice PDF.

API Routes

Method	Route	Description	Auth
GET	/api/plans	Get all plan details	Public
POST	/api/payment/create-order	Create Razorpay order	Auth
POST	/api/payment/verify	Verify payment signature server-side	Auth
POST	/api/payment/webhook	Razorpay webhook handler	Public
GET	/api/payment/history	Get user's payment history	Auth

Razorpay Integration Flow

1. User clicks 'Upgrade Plan' → Frontend calls POST /api/payment/create-order.
2. Backend creates Razorpay order via SDK → returns order_id + amount + currency.
3. Frontend opens Razorpay checkout modal with order details.
4. User completes payment → Razorpay returns razorpay_payment_id + razorpay_signature.
5. Frontend sends both to POST /api/payment/verify.
6. Backend verifies HMAC-SHA256 signature → updates User.plan + creates Subscription doc.
7. Razorpay also sends webhook to /api/payment/webhook for async confirmation.
8. Feature gating: all protected routes check req.user.plan before allowing access.

Implementation Plan

Phase	Task	Duration	Priority
Backend	Subscription.model.js + Razorpay SDK setup	0.5d	Critical
Backend	POST /payment/create-order + POST /payment/verify	1d	Critical
Backend	Webhook handler with HMAC signature verification	1d	Critical
Backend	Plan middleware: isPlanActive() guard on Workspace routes	0.5d	Critical
Frontend	Pricing page with 3 plan cards (highlighted recommended)	1d	High
Frontend	Razorpay checkout integration (rzp.js CDN)	1d	Critical
Frontend	Payment success/failure pages + invoice download	0.5d	High
Backend	Invoice PDF generation (pdfkit/puppeteer) + Cloudinary store	1d	Medium

Admin Panel

User Management · Expert Assignment · Plan Control

Admin Capabilities

User List	View all registered users with plan, status, business info. Search + filter + paginate.
Expert Assignment	Assign or reassign an EGCN expert to a client. Creates the chat room.
Plan Override	Manually set a user's plan and expiry date (comps, corrections).
Subscription Logs	View all payment transactions, statuses, failed payments.
Report Access	Admin can view any client's insight reports.
Manage Experts	Create expert accounts. Set expertise area (Marketing/Finance/CashFlow).

API Routes

Method	Route	Description	Auth
GET	/api/admin/users	List all users with pagination + filters	Admin
GET	/api/admin/users/:id	Get single user full profile	Admin
PATCH	/api/admin/users/:id/plan	Override user plan	Admin
POST	/api/admin/assign-expert	Assign expert to client	Admin
GET	/api/admin/experts	List all expert accounts	Admin
POST	/api/admin/experts	Create new expert account	Admin
GET	/api/admin/subscriptions	All subscription records	Admin
GET	/api/admin/stats	Platform-wide stats: users, revenue, plans	Admin

Implementation Plan

Phase	Task	Duration	Priority
Backend	authorise('admin') middleware + role check	0.5d	Critical
Backend	Admin user CRUD routes with pagination	1d	High
Backend	Expert assignment logic + room creation	0.5d	High
Frontend	Admin panel UI: sidebar with Users/Experts/Plans/Stats	1d	High
Frontend	Users table with search, filter, plan badge, actions	1d	High
Frontend	Assign Expert modal + plan override form	0.5d	High
Frontend	Platform stats dashboard (total users, revenue, plan split)	1d	Medium

Notification & Suggestion Engine

50-Message Pool · Smart Triggers · Email Alerts

Suggestion Message Pool

A pool of **50 pre-written advisory messages** is maintained in `src/constants/suggestions.js`. Messages are tagged with a performance tier:

Tier A — Below 50%	10 messages. Urgency + motivation. e.g. 'Your target is slipping — push harder today!'
Tier B — 50–80%	20 messages. On-track encouragement. e.g. 'Great pace! Keep the momentum going.'
Tier C — 80–100%	15 messages. Celebration + stretch goals. e.g. 'Outstanding! Can you hit 105% today?'
Tier D — >100%	5 messages. Exceptional performance celebration.

Notification Events

Daily Entry Reminder	If no daily entry by 6 PM → send push notification + email reminder.
Target Achievement	When user crosses 80% of monthly target → congratulatory notification.
Plan Expiry Warning	7 days before plan expires → email + in-app banner.
New Message	Real-time Socket.io notification when expert sends a chat message.
Meeting Reminder	1 hour before scheduled meeting → push + email notification.
Expert Assigned	When admin assigns expert to client → email to both parties.

Implementation Plan

Phase	Task	Duration	Priority
-------	------	----------	----------

Backend	Create 50 suggestion messages in constants + tier-weighting logic	1d	High
Backend	Cron job (node-cron) for daily reminder check at 6 PM IST	0.5d	High
Backend	Email templates (Nodemailer): reminder, expiry, achievement	1d	Medium
Frontend	In-app notification bell + notification list dropdown	1d	Medium
Frontend	Toast notifications for real-time socket events	0.5d	Medium
Backend	Meeting reminder cron — check 1hr before + send email	0.5d	Medium

Deployment & DevOps

Docker · Nginx · GitHub Actions · Cloud Hosting

Environment Variables

```
# server/.env
NODE_ENV=production
PORT=5000
MONGO_URI=mongodb+srv://...
JWT_ACCESS_SECRET=
JWT_REFRESH_SECRET=
REDIS_URL=redis://...
RAZORPAY_KEY_ID=rzp_live...
RAZORPAY_KEY_SECRET=...
CLOUDINARY_CLOUD_NAME=...
CLOUDINARY_API_KEY=...
CLOUDINARY_API_SECRET=...
SENDGRID_API_KEY=SG...
CLIENT_URL=https://egcn.in

# client/.env.local
VITE_API_URL=https://api.egcn.in
VITE_RAZORPAY_KEY_ID=rzp_live...
VITE_SOCKET_URL=https://api.egcn.in
```

Docker Compose Setup

```
services:
  client:
    build: ./client
    ports: ['3000:80'] # Nginx serves React build
  server:
    build: ./server
    ports: ['5000:5000']
    env_file: ./server/.env
  nginx:
    image: nginx:alpine
    ports: ['80:80', '443:443']
    volumes: [./nginx.conf:/etc/nginx/nginx.conf]
```

GitHub Actions CI/CD Pipeline

```
name: Deploy EGCN WebApp
on: push (branch: main)

jobs:
test:
  - Checkout code
  - npm ci (client + server)
  - Run ESLint
  - Run Jest unit tests

build:
  - Build React app (npm run build)
  - Build Docker images
  - Push to Docker Hub / GHCR

deploy:
  - SSH into VPS
  - docker-compose pull + up --build -d
  - Health check: curl https://api.egcn.in/health
```

Hosting Recommendations

Frontend (React)	Vercel (free tier) or Cloudflare Pages. Automatic deploys from main branch.
Backend (Express)	Railway.app or Render.com (managed Node). Or AWS EC2 / DigitalOcean Droplet.
MongoDB	MongoDB Atlas M0 (free) → M10 (paid) as user base grows.
Redis	Upstash Redis (serverless, free tier sufficient for start).
Media	Cloudinary free plan (25 GB storage, 25 GB bandwidth/month).
Video Calls	Daily.co (free 10,000 minutes/month) or self-hosted PeerJS server.
Domain + SSL	Cloudflare for DNS + free SSL. Point api.egcn.in to backend server.

Implementation Plan

Phase	Task	Duration	Priority
Setup	Dockerfile for client (nginx) + server (node) + docker-compose	1d	Critical
Setup	Nginx config: reverse proxy /api → server, / → React SPA	0.5d	Critical
CI/CD	GitHub Actions: lint + test + build + deploy workflow	1d	High
Deploy	Deploy to Vercel (frontend) + Railway (backend) for staging	0.5d	High

Deploy	Configure environment variables on hosting platforms	0.5d	Critical
Security	Rate limiting (express-rate-limit), Helmet.js, CORS config	0.5d	Critical
Monitor	Sentry error tracking (free tier) + Winston logs to file	0.5d	Medium

MODULE 12

Master Implementation Timeline

Sprint Plan · 12 Weeks · Effort Estimates

Sprint Plan (12 Weeks)

Phase	Task	Duration	Priority
Week 1	Project setup: Vite + Express + MongoDB + Tailwind + folder structure	5d	Critical
Week 1	Homepage: all sections (Hero, Tiles, Testimonials, Footer)	5d	Critical
Week 2	Auth: Signup + Signin + JWT + Redis + Email OTP	5d	Critical
Week 2	Protected routing + Redux auth slice + Axios interceptors	5d	Critical
Week 3	Dashboard layout: Sidebar + Header + Stat cards + API	5d	Critical
Week 3	MonthlyTarget: form + required daily calc + charts	5d	High
Week 4	Daily Input: form + charts + suggestion engine	5d	Critical
Week 4	Target Analysis page: 30-day trend + projection charts	5d	High
Week 5	Insights: Daily Report + Monthly Report pages + charts	5d	High
Week 5	Insights: Annual Report + BEP Report (inputs + calcs + charts)	5d	High
Week 6	Workspace: Chat UI + Socket.io real-time messaging	5d	Critical
Week 6	Workspace: Media upload (Cloudinary) + file preview in chat	5d	High
Week 7	Workspace: Video call (PeerJS/Daily.co) + Calendar UI	5d	High
Week 7	Plans & Pricing page + Razorpay integration + webhook	5d	Critical
Week 8	Admin Panel: Users list + Expert assignment + Plan override	5d	High
Week 8	Notification engine: crons + email templates + in-app bell	5d	Medium
Week 9	PDF export for reports (jsPDF + html2canvas)	3d	Medium
Week 9	Mobile responsiveness pass across all pages	4d	High
Week 10	Unit testing: services + controllers (Jest)	3d	High
Week 10	E2E testing: Auth + Target + Payment flows (Playwright)	4d	High

Week 11	Docker + Nginx + GitHub Actions CI/CD setup	3d	Critical
Week 11	Staging deploy + QA pass + bug fixing	5d	Critical
Week 12	Production deploy + DNS + SSL + monitoring setup	2d	Critical
Week 12	Performance optimisation: lazy loading, code splitting, caching	3d	High

Effort Summary

Total Development Time	~12 weeks (1 full-stack developer) / ~6 weeks (2-person team)
Frontend Effort	~25 days across 11 modules
Backend Effort	~20 days across 11 modules
Testing & QA	~7 days
DevOps & Deployment	~5 days
Total Estimated Days	~57 developer-days

Recommended Team Roles

Full-Stack Lead	Owns architecture, auth, DB schema, Socket.io, Razorpay. Node + React expert.
Frontend Developer	Owns all page UIs, chart components, Workspace UI, responsive design.
Backend Developer	Owns REST APIs, report aggregation, BEP engine, admin routes, crons.
DevOps (Part-time)	Docker, Nginx, CI/CD, hosting setup, SSL, monitoring.

This document covers the complete implementation plan for the EGCN WebApp across 11 functional modules. Each module includes schema design, API routes, business logic, component breakdown, and a sprint-level implementation plan. The project is scoped for a 12-week delivery timeline with a 2-person MERN team.

EGCN WebApp — Implementation Plan v1.0 — May 2026